DigitalSkola
Uncover The World of Digital Skills with Us

Sesi 4

Python & Fungsi Object Oriented Programming (OOP)

Akmal Fauzan

Today's Agenda

1. **Function & Return Types**
   Definisi, parameter, argument passing, dan return values.
2. **Intro to OOP**
   Paradigma OOP, 4 pilar utama, serta Class & Instance.
3. **Inheritance & Method**
   Pewarisan kelas, Instance Method, Class Method & Static Method.
4. **Type Hinting & Pydantic Dasar**
   Static typing di Python, validasi data, dan implementasi AI.

Profile Mentor



Akmal Fauzan, S.Stat He/Him

Data Scientist at Pancaran Group | A man behind Seara Data | Senior Data Analyst

Jakarta Metropolitan Area · Contact info

500+ connections

Experience

Data Scientist at Pancaran Group (Aug 2025 – Now)

Mentor at Digitalskola (Jun 2026 - Now)

- **Tutor Bootcamp Business Intellegence**
- **Tutor Bootcamp AI Engineer**
- **Tutor Bootcamp Data Analyst**
- **And more**

Founder at Seara Data (Feb 2026 - Now)

LinkedIn : linkedin.com/in/akmal-fauzan-

DigitalSkola
Uncover The World of Digital Skills with Us

Part 1.

Function & Return Types

What is Function?

Function adalah blok kode terisolasi yang menjalankan tugas spesifik dan dapat digunakan kembali (*reusable*).

Filosofi Utama: DRY

Kemampuan fungsi untuk digunakan berulang kali menjadikan fungsi sebagai standar utama penerapan prinsip **DRY (Do Not Repeat Yourself)**.



Blok Reusabel

Input → Proses Logika → Output Hasil

Why We Need Function?

Reusability

Menyediakan satu blok kode yang dapat dipanggil kembali kapan saja tanpa perlu menulis ulang instruksi yang sama.

Modularity & Readability

Membagi aplikasi menjadi komponen modular kecil yang rapi. Nama fungsi deskriptif meningkatkan keterbacaan kode.

Abstraction

Sembunyikan detail kompleksitas logika eksekusi di balik pemanggilan antarmuka fungsi yang sederhana.

Aturan Penulisan Fungsi

1. **Keyword def**
   Diawali kata kunci def
2. **Function Name**
   Nama unik (snake_case)
3. **Parenthesis () & :**
   Diakhiri tanda kurung & titik dua
4. **Indented Body**
   Logika dengan indentasi 4 spasi

```
def hitung_diskon(harga: int, persen: float) -> float:
    # Body fungsi dengan indentasi
    total_diskon = harga * persen
    return harga - total_diskon
```

Parameters & Argument Passing

TANPA PARAMETER

Statis / Tetap

```
def add():
    result = 5 + 10
    return result

# Pemanggilan
add() # Selalu menghasilkan 15
```

DENGAN PARAMETER

Dinamis / Fleksibel

```
def add(a: int, b: int):
    result = a + b
    return result

# Pemanggilan argumen
add(5, 6) # Output: 11
```

Function Output (Return vs None)

Secara default, fungsi Python mengembalikan objek None jika kata kunci return tidak ditentukan eksplisit.

Return None (Default)

```
def add():
    result = 5 + 10

a = add()
print(a) # Output: None
type(a)  #
```

Return Result Explicit

```
def add(a: int, b: int):
    result = a + b
    return result

a = add(5, 6)
print(a) # Output: 11
type(a)  #
```

DigitalSkola
Uncover The World of Digital Skills with Us

Part 2.

Intro to Object Oriented Programming

PARADIGMA PEMROGRAMAN

Object Oriented Programming (OOP)

OOP adalah paradigma pemrograman yang mengorganisasi dan mengelompokkan kode ke dalam unit terpadu yang disebut **Object**.

What OOP is Good For?

Real World Model

Memudahkan pemetaan antara entitas dunia nyata (User, Transaction, Product, Model) menjadi objek program.

Scalability

Memastikan struktur kode tetap rapi dan terorganisir meskipun aplikasi berkembang hingga ribuan baris kode.

Maintenance

Memudahkan pencarian dan perbaikan bug secara tepat target pada modul Class yang bermasalah.

4 Main Concepts of OOP

Encapsulation

Membungkus data (attribute) & method dalam satu unit (class) serta membatasi akses langsung dari luar.

Inheritance

Kemampuan child class untuk mewarisi attribute dan method dari parent class secara efisien.

Polymorphism

Kemampuan objek memberikan respon atau perilaku berbeda terhadap pemanggilan method yang sama.

Abstraction

Menyembunyikan detail implementasi internal yang rumit dan hanya menampilkan antarmuka sederhana.

Class & Instance

1. Class (Blueprint)

Cetak biru objek yang mendefinisikan atribut & method.

```
class Mobil:
    def __init__(self, merk: str):
        self.merk = merk

    def horn(self):
        return "Wok wok!"
```

2. Instance (Real Object)

Wujud nyata objek yang diinisialisasi dari Class.

```
# Instansiasi objek
toyota = Mobil(merk="Toyota")

# Memanggil method objek
toyota.horn() # Output: Wok wok!
```

DigitalSkola
Uncover The World of Digital Skills with Us

Part 3.

Inheritance & Method

Inheritance (Pewarisan)

Inheritance memungkinkan sebuah Class baru (Child Class) mewarisi atribut dan method dari Class yang sudah ada (Parent Class).

Catatan Aksesibilitas:

Hampir seluruh atribut dan method parent class dapat diakses secara bebas oleh child class, kecuali anggota yang bersifat private.

Keuntungan Inheritance

1. Reusability

Mencegah duplikasi kode berulang dengan memanfaatkan anggota yang telah didefinisikan pada parent class.

2. Hierarchy

Membuat hirarki hubungan logis antar objek (misal: Bougenville IS-A Flower).

3. Extensibility

Child class dapat menambahkan atribut atau method spesifik baru tanpa mengganggu fungsi parent class.

4. Polymorphism

Memungkinkan pembuatan variasi child class dengan perilaku unik dari antarmuka parent yang sama.

Konsep Method dalam Class

Method adalah fungsi yang didefinisikan di dalam sebuah Class dan merepresentasikan perilaku atau kemampuan dari Class tersebut.

Perbedaan 1

Method selalu terikat (bound) pada struktur Class atau Object tertentu.

Perbedaan 2

Method dapat mengakses & memanipulasi data (attribute) internal milik Class/Instance.

3 Tipe Method pada Python

1

Instance Method

Tipe paling umum. Menggunakan self sebagai parameter pertama untuk mengakses atribut objek setelah diinstansiasi.

2

Class Method

Terikat pada level Class menggunakan parameter cls dan decorator @classmethod.

3

Static Method

Tidak terikat pada objek maupun class. Menggunakan decorator @staticmethod untuk logika mandiri.

DigitalSkola
Uncover The World of Digital Skills with Us

Part 4.

Type Hinting & Pydantic Dasar

Apa itu Type Hinting?

Type Hinting adalah pertanda atau informasi eksplisit mengenai tipe data pada variabel, parameter fungsi, maupun return value.

Tanpa Type Hinting

```
duck_count = 10
```

Dengan Type Hinting

```
duck_count: int = 10
```

Mengapa Butuh Type Hinting?

1

Readability

Memudahkan pengembang memahami konteks dan struktur tipe data yang sedang diproses dalam kode.

2

IDE Support

Meningkatkan kemampuan autocomplete dan saran kode pada IDE (VS Code, PyCharm).

3

Bug Detection

Mendeteksi kesalahan tipe data lebih awal menggunakan linter seperti mypy sebelum program dijalankan.

What is Pydantic?

Pydantic adalah pustaka (library) Python yang memaksakan dan mengaktifkan aturan type hinting secara nyata pada saat runtime.

Mengapa Menggunakan Pydantic?

Secara default, type hinting di Python bersifat dokumentatif dan tidak melakukan verifikasi saat runtime. Pydantic melakukan otomatisasi parsing dan validasi tipe data secara ketat.

Studi Kasus: Standard Class vs Pydantic

Python Class Biasa (String dipaksa)

```python
class Flower:
    def __init__(self, name, color, height):
        self.height = height

b = Flower("Rose", "Red", "5.0")
# Output: height tetap
```

Pydantic BaseModel (Parsing Otomatis)

```python
from pydantic import BaseModel

class Flower(BaseModel):
    height: float

b = Flower(name="Rose", height="5.0")
# Output: height terkonversi ke
```

Pydantic: Runtime ValidationError

Ketika input data tidak sesuai dan tidak dapat di-parse, Pydantic akan menghentikan eksekusi dan memberikan rincian error yang jelas.

```
# Input nilai 'five' yang invalid untuk float
b = Flower(name="Rose", color="Red", height="five")
```

ValidationError Output:

```
1 validation error for Flower
 height -> Input should be a valid number, unable to parse string as a number [type=float_parsing]
```

DigitalSkola

Bootcamp AI Engineer

Penerapan Pydantic pada AI Engineering

Dalam pengembangan AI dan LLM Agent, skema data yang terstruktur sangatlah krusial. Pydantic digunakan secara luas pada framework seperti **LangChain** dan **FastAPI** untuk menjamin output LLM mematuhi format JSON yang terstruktur.



Structured Output LLM

Menjamin Agent & Prompt AI mengembalikan format data presisi tanpa hallucination struktur.



Praktik & Sesi Tanya Jawab

Terima Kasih atas partisipasi anda pada sesi Python & Fungsi OOP!

- Functions & OOP Basic
- Inheritance
- Pydantics

DigitalSkola
Uncover The World of Digital Skills with Us

Thank You.

Sampai Jumpa di Sesi Selanjutnya!

LinkedIn : linkedin.com/in/akmal-fauzan-